

Aktivitas Percobaan

Perbandingan Imperatif dan Deklaratif dalam Perubahan Teks

2. Objek TextView (variabel tvMessage) diubah langsung melalui propertinya

```
tvMessage.text = "Tombol sudah diklik"
```

3. Yang diubah adalah state message, bukan komponen UI secara langsung

```
message = "Tombol sudah diklik"
```

Compose lalu otomatis melakukan recomposition pada `Text(text = message, ...)` karena state-nya berubah.

4. Karena Compose tidak memegang referensi langsung ke objek UI seperti TextView pada XML/View. Compose bersifat deklaratif, hanya mendeskripsikan tampilan berdasarkan state saat ini. Compose sendiri yang bertanggung jawab merender ulang bagian UI yang bergantung pada state tersebut.

5. Ketika sebuah event terjadi seperti saat tombol `btnChange` diklik aksi tersebut memicu fungsi `setOnClickListener`. Melalui view listener ini, sistem langsung mengakses objek TextView (`tvMessage`) untuk mengubah properti teksnya secara eksplisit. Karena properti View tersebut dimodifikasi secara langsung, sistem Android akan merender ulang komponen tersebut sehingga teks baru langsung diperbarui dan tampak di layar saat itu juga.

6. Ketika sebuah tombol diklik, aksi tersebut memicu fungsi lambda `onClick` yang bertugas mengubah nilai dari variabel `message` (sebuah `mutableStateOf`). Karena Jetpack Compose memantau setiap perubahan status, Compose akan langsung mendeteksi perubahan nilai pada state yang dibaca oleh komponen `Text`. Akibatnya, Compose menjadwalkan dan menjalankan ulang proses recomposition pada bagian UI yang bergantung pada state tersebut. Hasil dari recomposition ini kemudian menghasilkan tampilan UI baru yang menampilkan teks yang sudah diperbarui secara real-time.

7. Meskipun tombol diklik berkali-kali dan nilai variabelnya berubah di dalam memori, teks pada layar tetap tidak akan berubah. Hal ini terjadi karena Jetpack Compose hanya akan memicu recomposition jika ia membaca sebuah State (yaitu objek `MutableState` hasil dari `mutableStateOf` yang aktif diobservasi oleh sistem). Variabel biasa tidak memiliki kemampuan observasi ini, sehingga perubahannya tidak akan pernah memberi tahu Compose untuk menggambar ulang UI. Selain itu, jika variabel tersebut tidak dibungkus dengan fungsi

remember, nilainya akan selalu ter-reset ke kondisi awal setiap kali recomposition terjadi karena fungsi composable akan dieksekusi ulang dari atas ke bawah.

8. Dalam pengembangan aplikasi, pendekatan imperatif berfokus pada langkah-langkah "apa yang harus dilakukan", di mana program secara eksplisit memerintahkan objek UI untuk berubah satu per satu (seperti mengeksekusi `tvMessage.text = ...`). Sebaliknya, pendekatan deklaratif bergeser pada cara berpikir tentang kondisi tampilan seharusnya berdasarkan data atau state saat ini. Pada model deklaratif ini, kita tidak lagi memerintah komponen UI secara langsung, melainkan mendeskripsikan hasil akhir tampilan yang diinginkan dan membiarkan framework seperti Jetpack Compose yang bekerja di balik layar untuk menentukan cara memperbarui UI tersebut secara efisien.

Event Handling State

2. Ketika tombol "Tambah" ditekan menggunakan variabel biasa, data di dalam memori sebenarnya berhasil diperbarui secara normal. Hal ini dibuktikan oleh LogCat yang secara konsisten menampilkan pesan "Counter berubah : X", di mana nilai X terus bertambah (1, 2, 3, dst.) setiap kali tombol diklik.

3. Sebaliknya, kondisi di layar justru berbanding terbalik dengan data di memori. Teks "Counter : 0" yang tampil pada layar aplikasi sama sekali tidak bergerak atau berubah, melainkan tetap mematum menampilkan angka 0 meskipun tombol telah diklik berkali-kali.

4. Masalah ini terjadi karena variabel count tersebut hanyalah variabel lokal biasa dan bukan merupakan sebuah State. Akibatnya, Jetpack Compose tidak memiliki kemampuan untuk "mengamati" perubahan nilainya. Tanpa adanya objek State, tidak ada sinyal atau mekanisme yang memberi tahu sistem bahwa UI perlu digambar ulang, sehingga proses recomposition tidak terpicu dan layar tetap mengunci tampilan lama.

9. Setelah variabel tersebut diubah menggunakan state, perilaku di balik layar tetap berjalan lancar. LogCat tetap memunculkan log "Counter berubah : X" yang nilainya terus meningkat secara berurutan setiap kali ada interaksi klik dari pengguna.

10. Perubahan besar terjadi pada antarmuka pengguna, di mana teks "Counter : X" pada layar kini berubah secara dinamis secara real-time. Angka yang ditampilkan di layar selalu akurat dan langsung menyesuaikan dengan jumlah klik yang dilakukan.

11. Hal ini bisa terjadi karena variabel count kini telah dideklarasikan sebagai sebuah State (`mutableStateOf`) yang aktif diobservasi oleh Compose. Setiap kali nilainya berubah, Compose otomatis mendeteksi komponen yang membaca data tersebut yaitu `Text("Counter :`

\$count") dan langsung memicu recomposition. Hal ini sangat berbeda dengan percobaan pertama di mana variabel biasa benar-benar diabaikan oleh sistem pembaruan UI Compose.

12. Recomposition merupakan proses di mana Jetpack Compose memanggil dan menjalankan ulang fungsi composable atau bagian spesifik di dalamnya yang bergantung pada suatu state ketika nilai state tersebut mengalami perubahan. Proses ini krusial agar tampilan UI selalu mencerminkan data paling aktual di memori. Menariknya, Compose mengeksekusi ini secara cerdas dan efisien, yaitu hanya menggambar ulang komponen yang terpengaruh langsung tanpa harus memuat ulang seluruh layar.

15. Ketika pengguna mengetik di dalam kolom input, LogCat akan langsung mencetak log "Text berubah : X" setiap kali ada karakter yang masuk atau dihapus. Nilai X ini merefleksikan isi teks paling mutakhir dari kolom tersebut yang ditangkap secara instan oleh fungsi onValueChange.

16. Pada layar aplikasi, komponen OutlinedTextField mampu merespons input dengan sangat sensitif. Karakter yang diketik oleh pengguna langsung muncul secara real-time dan teks di dalam kolom langsung berubah sesuai dengan ketikan tangan pengguna.

17. Perilaku responsif ini digerakkan oleh event onValueChange pada OutlinedTextField yang langsung aktif begitu ada perubahan input. Event ini bertugas memperbarui nilai state text. Karena properti value pada komponen field diikat langsung ke state tersebut (value = text), perubahan data yang instan ini langsung memicu recomposition lokal sehingga karakter baru yang diketik langsung memantul dan tampil di layar.

18.

- count (bertipe Int): Berfungsi untuk melacak dan menyimpan total akumulasi klik pada tombol "Tambah".

- text (bertipe String): Berfungsi untuk menampung seluruh karakter teks terkini yang ada di dalam Text Field.

19.

- onClick yang terpasang pada komponen tombol "Tambah".

- onValueChange yang terpasang pada komponen input OutlinedTextField.

20. onClick akan dieksekusi secara instan setiap kali pengguna menyentuh atau menekan tombol "Tambah". onValueChange akan berjalan di latar belakang setiap kali terjadi perubahan nilai pada Text Field, baik saat pengguna menambah karakter baru, menghapus tulisan, maupun memodifikasi teks yang sudah ada.

Implicit Intent

11. `Intent.ACTION_VIEW` menampilkan/membuka data kepada pengguna dengan aplikasi yang paling sesuai untuk data tersebut. Sifatnya umum; jenis data (URI) yang menentukan aplikasi apa yang akan dibuka (browser untuk URL `http`, aplikasi Maps untuk data `geo`;, dll).

`Intent.ACTION_DIAL` membuka aplikasi telepon/dialer dengan nomor yang sudah terisi otomatis, tanpa langsung melakukan panggilan (pengguna masih harus menekan tombol panggil secara manual). Ini berbeda dengan `ACTION_CALL` yang langsung memanggil.

`Intent.ACTION_SENDTO` digunakan untuk mengirim pesan/data ke tujuan tertentu tanpa lampiran besar (misalnya membuka aplikasi email dengan alamat tujuan yang sudah terisi via skema `mailto`); cocok dipasangkan dengan `Intent.createChooser()` agar pengguna bisa memilih aplikasi email yang diinginkan.

16. Fungsi data pada Intent

Properti data (berupa objek `Uri`) berfungsi menentukan konten atau tujuan spesifik yang ingin diakses oleh Intent tersebut misalnya alamat website (`https://...`), nomor telepon (`tel:...`), alamat email (`mailto:...`), atau koordinat lokasi (`geo:...`). Skema URI inilah yang memberi tahu sistem jenis data apa yang sedang diminta.

17. Hubungan Action dan Data terhadap pilihan aplikasi yang dimunculkan sistem Android

Sistem Android mencocokkan kombinasi Action + tipe Data (skema URI) dengan Intent Filter yang didaftarkan oleh aplikasi-aplikasi yang terpasang di perangkat. Action menyatakan "apa yang ingin dilakukan" (melihat, menghubungi, mengirim, dst.), sedangkan Data menyatakan "terhadap konten seperti apa" (URL, nomor telepon, alamat email, koordinat). Sistem hanya akan menawarkan aplikasi-aplikasi yang intent filter-nya cocok dengan pasangan Action dan skema Data tersebut itulah sebabnya `ACTION_VIEW` dengan `geo`: membuka aplikasi Maps, sedangkan `ACTION_VIEW` dengan `https://` membuka browser, meski Action-nya sama.